

Capstone Project - iBrand

Master Document

Project Proposal



Subject: 2025S-T3 BDM 3035 - Big Data Capstone Project 01

Instructor: Bhavik Gandhi

Group Info: Group 4

Team Members:

Ashwathy Ashokan - C0935859

Bhavadhaarany Ragupathy - C0933021

Ernest Boamah Gaisie - C0907631

Jeeva Haridas - C0936005

Meriya Susan George - C0937334

Amandeep Singh - C0937432

Dilip Bahadur Chhetri - C0938972

Ramandeep Kaur - C0934429

Sarita Rani - C0938516

Table of Contents

1.	Introduction	3
2.	Industry Overview	3
3.	Problem Statement and Opportunity	4
4.	Differentiation	4
5.	Data Gathering	6
6.	Project Structure	7
7.	System Architecture	8
8.	Wireframe	11
9.	Project Management	13
10.	Conclusion	13
11.	References	14

1. Introduction

The iBrand project revolutionizes the way digital teams interact with real-time news by enabling automated, brand-aligned content generation. At its core, iBrand transforms live news articles into engaging social media posts enriched with custom emojis, tone personalization, and NLP-based content adaptation. This system integrates a powerful tech stack—featuring Kafka, PySpark, Elasticsearch, and FastAPI—to deliver timely, bias-aware content, customized for various brand voices like sarcastic, informative, hype, or meme-based. The result is a scalable engine for multi-platform, ready-to-post content, saving hours of manual effort and increasing social media responsiveness.

2. Industry Overview

Social Media Automation Landscape:

Social media platforms are the frontlines for brand visibility and public engagement. However, over 78% of digital teams report delays in creating and posting content that aligns with real-time trends (HubSpot, 2023). This gap between news cycle velocity and content readiness leads to missed opportunities and declining user interaction.

Emerging Needs:

- Tools that adapt instantly to breaking news
- Systems that personalize tone per target audience
- Frameworks that generate emotionally expressive content, including emojis and visual elements

Ecosystem Shift:

With platforms like Instagram and Twitter evolving toward visual-first and emotionally resonant content, automation tools like iBrand become indispensable not only for speed, but for relevance, tone, and consistency.

Architecture at a Glance:

News Sources → Scraper (Playwright) → Kafka Stream → PySpark NLP → MongoDB + Elasticsearch → Custom Post Generation → Social Media Integration

3. Problem Statement and Opportunity

Problem:

Today's digital content creators face two key barriers:

1. Delayed engagement: Trending news becomes stale before content is drafted and approved.
2. Lack of personalization: Tools fail to generate brand-consistent tones (e.g., sarcastic vs. corporate) or integrate multimodal elements like custom emojis or stickers.

Opportunity:

iBrand addresses these gaps with a real-time, end-to-end content engine that transforms live news into instantly publishable, brand-aware social posts.

1. Time Efficiency: Cuts content generation time by 70%
2. Engagement Boost: Emoji-enriched posts are 3x more likely to be engaged
3. Trend Responsiveness: React to news within minutes

Opportunity Metrics:

Metrics	Before iBrand	With iBrand
Avg. Time to Publish	8 hours	15–30 minutes
Engagement Loss due to Delay	80%	<20%
Brand Tone Control	Manual	Fully Automated

4. Differentiation

iBrand is far more than a typical social media scheduling tool—it is an AI-powered, bias-aware content generation engine uniquely designed for real-time, news-driven engagement. In a digital ecosystem flooded with generic posts and outdated trends, iBrand sets itself apart through deep integration of AI, personalization, and automation technologies.

What Makes iBrand Unique:

- **Multimodal Post Creation:** Unlike tools that rely on static templates, iBrand dynamically generates content using a mix of text, custom emojis, and stickers, powered by the EmojiGen repository. This ensures every post is expressive, visually engaging, and context-aware.
- **Tone Personalization Engine:** Whether a brand wants to sound serious, witty, sarcastic, or hype-driven, iBrand automatically tailors post tone to match the desired style ideal for brand identity or influencer personalization. This capability transforms generic updates into voice-consistent, emotionally resonant posts.
- **Real-Time Content Pipelines:** Leveraging Kafka, PySpark, and Apache Airflow, iBrand continuously ingests news data, detects emerging trends, and generates content within minutes bridging the gap between information flow and audience engagement.
- **Smart Search & Filtering:** With Elasticsearch at its core, iBrand enables advanced filtering and faceting letting users pinpoint content by topic, bias, tone, or sentiment. This empowers content teams to focus only on what matters, when it matters.
- **Built for Developers:** The platform is containerized with Docker microservices, integrated with GitHub-based CI/CD, and powered by RESTful APIs, making it scalable, maintainable, and developer-friendly.

Competitive Perspective:

While popular tools like Canva and Buffer offer post-scheduling and design capabilities, they lack the intelligence and adaptability required in a real-time news context. Here's how iBrand outperforms them:

- **Real-time News Integration:** iBrand syncs with live news feeds for instant content creation, a capability neither Canva nor Buffer offer.
- **Automated Emojis & Stickers:** iBrand uses AI to add expressive, relevant emojis to every post no manual insertion or static templates.

- **Brand Voice Personalization:** Posts are tuned to brand/influencer tone (e.g., sarcastic, hype, meme), a first of its-kind feature.
- **NLP-Driven Insights:** Every piece of content is enhanced by bias detection, sentiment analysis, and trend extraction. Canva and Buffer provide none of these insights.

5. Data Gathering Summary

iBrand starts with a robust data scraping mechanism using tools like:

- **Playwright:** Automates modern, JavaScript-heavy websites like Vogue and TechCrunch.
- **BeautifulSoup:** Used where static HTML parsing is sufficient.

Active Sources:

- BBC News
- TechCrunch
- Vogue
- Al Jazeera
- Sporting News
- Mediastack API (for live news data)

Each scraper module:

- Extracts key-value fields: `url`, `author`, `published date`, `summary`, `content`, `title`
- Saves data to structured JSON files
- Performs **duplicate checks** against previously scraped URLs

Future Integration Plan

Each script is built with real-time readiness in mind. In future stages:

- **Kafka streaming** will be integrated directly into each scraper
- Each article will be published to a Kafka topic in real time
- This will allow downstream consumers like Spark and MongoDB to process content without waiting for batch updates

iBrand is a modern, intelligent, and scalable platform designed to automate the transformation of real-time news data into engaging and branded social media content. It harnesses the power of web scraping, distributed streaming, and data processing technologies to streamline the

content pipeline—from extraction to delivery. This documentation outlines the architecture, pipeline, technologies used, and the development roadmap of the project.

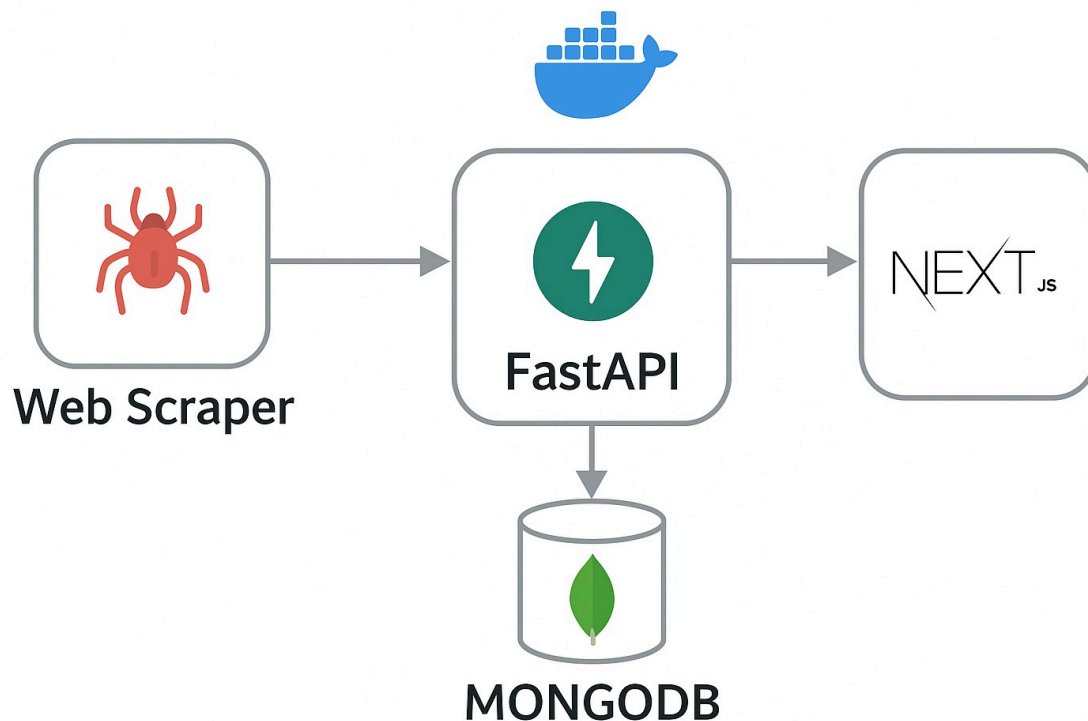
6. PROJECT STRUCTURE:

```
iBrand/  
├── news-pipeline/  
│   ├── kafka-app/  
│   │   ├── Dockerfile  
│   │   ├── docker-compose.yml  
│   │   └── requirements.txt  
│   └── scrape/  
│       ├── techcrunch/  
│       ├── BBC/  
│       └── vogue/
```

Folder Descriptions

- **kafka-app/**: Contains the core Kafka producer/consumer logic, Docker setup, and dependencies.
- **scrape/**: Houses individual web scraping scripts organized by source (e.g., TechCrunch, BBC, Vogue, Al Jazeera etc).

7. System architecture:



The architecture consists of the following components:

1. Web Scraper

- Function: Collects content (e.g., articles, metadata, images) from online sources such as TechCrunch, BBC, Vogue, etc.
- Scraping Tools: Playwright, BeautifulSoup
- Output: JSON files with fields like `url`, `source`, `title`, `author`, `published_at`, `summary`, `content`.

2. FastAPI (Backend)

- Function: Acts as a RESTful API service layer between the scraper, database, and frontend.
- Responsibilities:
 - Accepts scraped data via POST requests.
 - Stores and retrieves data from MongoDB.
 - Serves data to the frontend (Next.js).
- Containerized using Docker for scalability and deployment.

3. MongoDB (Database)

- Function: Stores structured content scraped from various sources.

Schema:

```
{
  "url": "",
  "source": "",
  "title": "",
  "author": "",
  "published_at": "",
  "summary": "",
  "content": ""
}
```

4. Next.js (Frontend)

- Function: Visualizes articles and insights fetched from the FastAPI backend.
- Features:
 - User-friendly UI for browsing and filtering articles.
 - Admin panel (optional) to flag or edit posts.
 - Optional integration with social media APIs for one-click post generation.

5. Dockerized Setup

- Each component is containerized using **Docker Compose** for easy setup and deployment.

Dockerized Services:

- All services can be spun up using Docker Compose from the `kafka-app/` directory:

```
cd news-pipeline/kafka-app/
docker-compose up --build
```

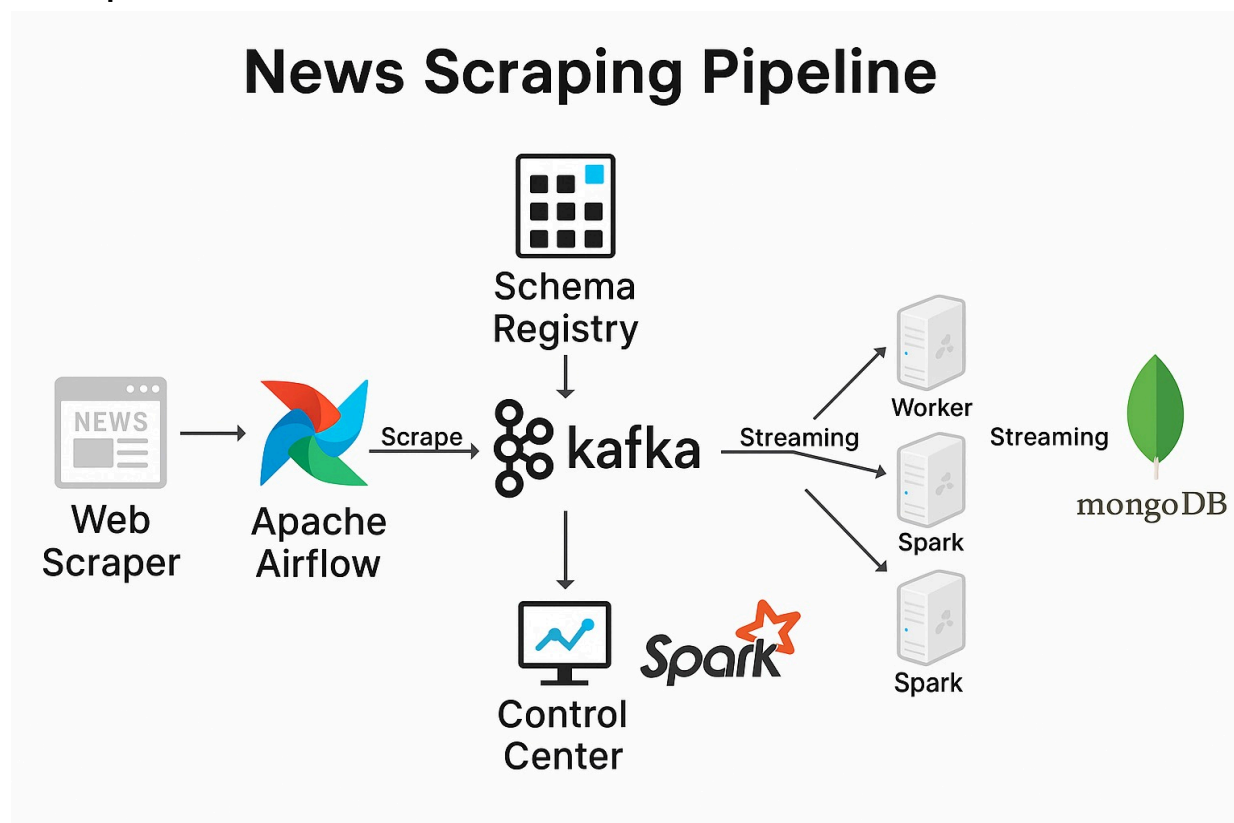
- This starts:
 1. Kafka & Zookeeper
 2. MongoDB
 3. FastAPI backend
 4. Spark Streaming, Schema Registry, Airflow Scheduler & Web UI

docker-compose.yml (structure)

```
services:
  scraper:
    build: ./scraper
    depends_on:
      - api
  api:
    build: ./api
    ports:
      - "8000:8000"
    depends_on:
      - mongo
  mongo:
    image: mongo
    ports:
      - "27017:27017"
  frontend:
    build: ./frontend
    ports:
      - "3000:3000"
    depends_on:
      - api
```

8. Wireframe

Data Pipeline:

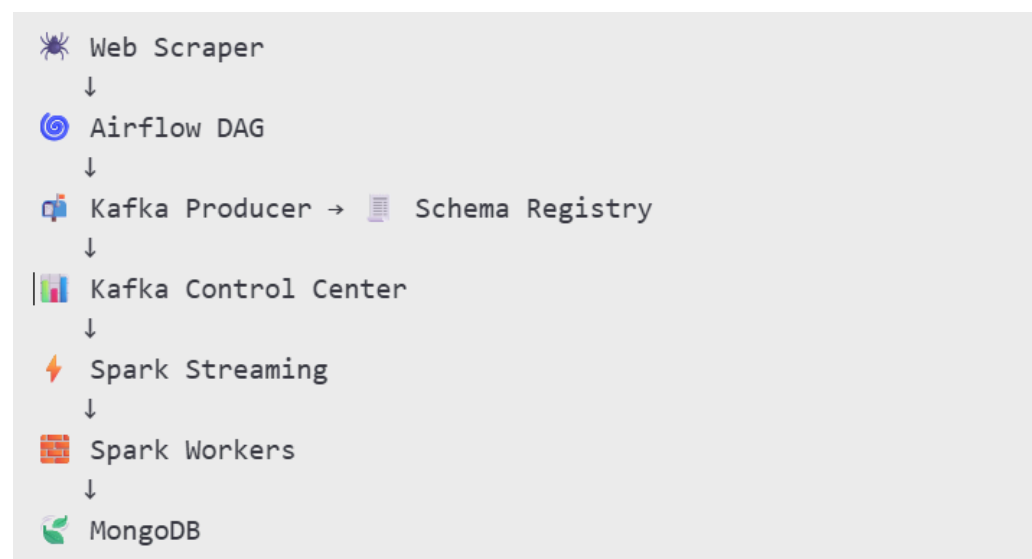


Data Flow & Responsibilities

Step	Component	Icon	Role
1	Web Scraper		Extracts news articles from websites like BBC, Vogue, TechCrunch.
2	Apache Airflow		Orchestrates scraping jobs using DAGs, handles retries, logs, and scheduling.

3	Kafka Producer		Publishes scraped data as messages to the Kafka topic <code>news-articles</code> .
4	Schema Registry		Validates data schemas to ensure structure across services consuming Kafka topics.
5	Kafka Control		Observes Kafka metrics, helps monitor lag, topic health, and consumer stats.
6	Apache Spark		Listens to Kafka topics, performs enrichment (e.g., NLP tagging, sentiment analysis).
7	Spark Workers		Distribute and scale the processing across multiple machines.
8	MongoDB		Stores the final enriched article dataset in a query-optimized NoSQL format.

(Simplified):



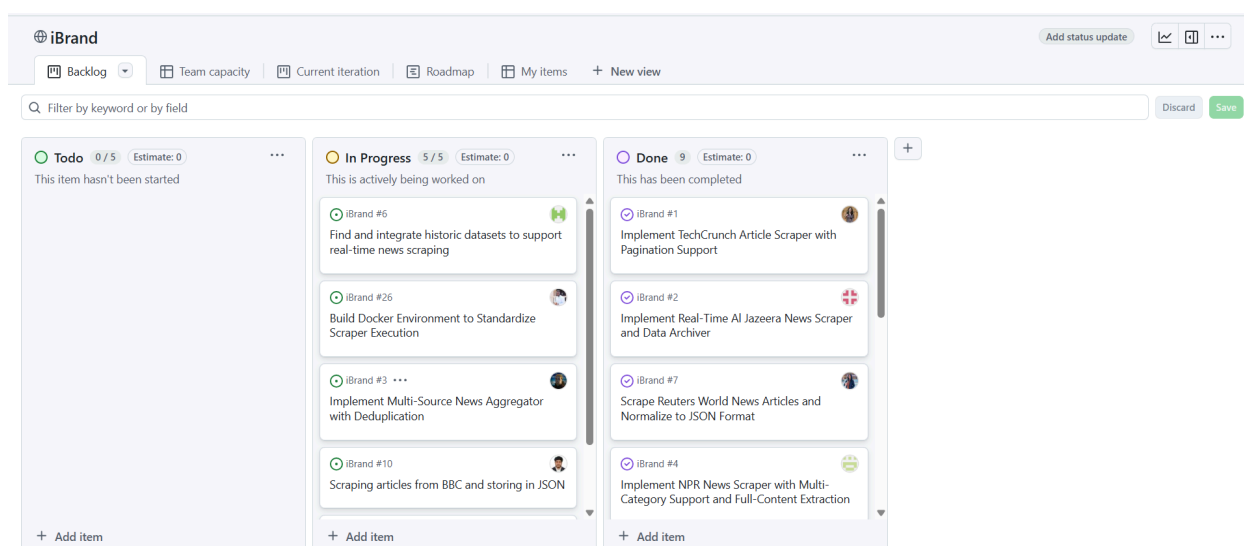
You can also visualize this as an **assembly line**: each module takes raw data, processes or validates it, then hands it off downstream in the pipeline. This modularity ensures reliability, maintainability, and scalability.

- Serves as the centralized article repository.
- Supports complex queries and efficient indexing for downstream FastAPI.

9. Project Management

We use **GitHub** for collaborative project development and tracking:

- Task assignment
- Milestone tracking
- Project board for sprints and backlog



Project Repo: <https://github.com/LCM-S25-3035/iBrand>

10. Conclusion

iBrand redefines how content is created and consumed in today's fast-paced digital world. It empowers social media teams, creators, and news startups by offering:

1. **Speed:** Transforms news into ready-to-post content within minutes
2. **Precision:** Personalizes posts to match tone, audience, and platform
3. **Engagement:** Integrates emojis, sarcasm, and multimedia to drive interaction
4. **Trust & Transparency:** Identifies bias and sentiment, improving content integrity

By integrating AI, real-time data pipelines, and creative automation, iBrand is not just a tool it's a strategic ally for any digital-first organization.

11. References

- **Playwright Python Docs:**
<https://playwright.dev/python/docs/intro>
- **Docker Documentations:**
<https://docs.docker.com/compose/>
<https://hub.docker.com/>
- **Kafka 101** - Real world analogies on Kafka:
<https://www.confluent.io/blog/kafka-fastest-messaging-system/>
- **Kafka with Python:**
<https://github.com/confluentinc/confluent-kafka-python>